CODE:

In [121…

```python
import pandas as pd
import numpy as np
from scipy.stats import zscore
from tqdm import tqdm
```

In [122…

```python
# File paths for omics data
omics_files = {
    "methylome_child": "clean_methylome_childhood.csv",
    "methylome_preg": "clean_methylome_pregnancy.csv",
    "mirna_child": "clean_mirna_childhood.csv",
    "mirna_preg": "clean_mirna_pregnancy.csv",
    "proteins_child": "clean_proteins_childhood.csv",
    "proteins_preg": "clean_proteins_pregnancy.csv",
    "metabolites_child": "clean_serum_metabolites_childhood.csv",
    "metabolites_preg": "clean_serum_metabolites_pregnancy.csv",
    "transcript_child": "clean_transcriptome_childhood.csv",
    "transcript_preg": "clean_transcriptome_pregnancy.csv"
}

# Load all omics data into DataFrames
omics_data = {name: pd.read_csv(path) for name, path in omics_files.items()}

# Show column names for base exposure data
print("\n👶 Exposure Base File Columns (1302 kids):")
print(base.columns.tolist())

# ✅ Display column names for each omics dataset
for name, df in omics_data.items():
    print(f"\n🔬 {name} Columns:")
    print(df.columns.tolist())
```

```
👶 Exposure Base File Columns (1302 kids):
['ID', 'h_abs_ratio_preg_Log', 'h_no2_ratio_preg_Log', 'h_pm10_ratio_preg_None', 'h_pm25_ratio_preg_None', 'hs_no2_dy_hs_h_Log', 'hs_no2_wk_hs_h_Log', 'hs_no2_yr_hs_h_Log', 'hs_pm10_dy_hs_h_None', 'hs_pm10_wk_hs_h_None', 'hs_pm10_yr_hs_h_None',
'hs_pm25_dy_hs_h_None', 'hs_pm25_wk_hs_h_None', 'hs_pm25_yr_hs_h_None', 'hs_pm25abs_dy_hs_h_Log', 'hs_pm25abs_wk_hs_h_Log', 'hs_pm25abs_yr_hs_h_Log', 'h_accesslines300_preg_dic0', 'h_accesspoints300_preg_Log', 'h_builtdens300_preg_Sqrt', 'h_conni
nd300_preg_Sqrt', 'h_fdensity300_preg_Log', 'h_frichness300_preg_None', 'h_landuseshan300_preg_None', 'h_popdens_preg_Sqrt', 'h_walkability_mean_preg_None', 'hs_accesslines300_h_dic0', 'hs_accesspoints300_h_Log', 'hs_builtdens300_h_Sqrt', 'hs_con
nind300_h_Log', 'hs_fdensity300_h_Log', 'hs_landuseshan300_h_None', 'hs_popdens_h_Sqrt', 'hs_walkability_mean_h_None', 'hs_accesslines300_s_dic0', 'hs_accesspoints300_s_Log', 'hs_builtdens300_s_Sqrt', 'hs_connind300_s_Log', 'hs_fdensity300_s_Lo
g', 'hs_landuseshan300_s_None', 'hs_popdens_s_Sqrt', 'h_Absorbance_Log', 'h_Benzene_Log', 'h_NO2_Log', 'h_PM_Log', 'h_TEX_Log', 'e3_alcpreg_yn_None', 'h_bfdur_Ter', 'h_cereal_preg_Ter', 'h_dairy_preg_Ter', 'h_fastfood_preg_Ter', 'h_fish_preg_Te
r', 'h_folic_t1_None', 'h_fruit_preg_Ter', 'h_legume_preg_Ter', 'h_meat_preg_Ter', 'h_pamod_t3_None', 'h_pavig_t3_None', 'h_veg_preg_Ter', 'hs_bakery_prod_Ter', 'hs_beverages_Ter', 'hs_break_cer_Ter', 'hs_caff_drink_Ter', 'hs_dairy_Ter', 'hs_fast
food_Ter', 'hs_KIDMED_None', 'hs_mvpa_prd_alt_None', 'hs_org_food_Ter', 'hs_pet_cat_r2_None', 'hs_pet_dog_r2_None', 'hs_pet_None', 'hs_proc_meat_Ter', 'hs_readymade_Ter', 'hs_sd_wk_None', 'hs_total_bread_Ter', 'hs_total_cereal_Ter', 'hs_total_fis
h_Ter', 'hs_total_fruits_Ter', 'hs_total_lipids_Ter', 'hs_total_meat_Ter', 'hs_total_potatoes_Ter', 'hs_total_sweets_Ter', 'hs_total_veg_Ter', 'hs_total_yog_Ter', 'hs_dif_hours_total_None', 'hs_as_c_Log2', 'hs_as_m_Log2', 'hs_cd_c_Log2', 'hs_cd_m
_Log2', 'hs_co_c_Log2', 'hs_co_m_Log2', 'hs_cs_c_Log2', 'hs_cs_m_Log2', 'hs_cu_c_Log2', 'hs_cu_m_Log2', 'hs_hg_c_Log2', 'hs_hg_m_Log2', 'hs_mn_c_Log2', 'hs_mn_m_Log2', 'hs_mo_c_Log2', 'hs_mo_m_Log2', 'hs_pb_c_Log2', 'hs_pb_m_Log2', 'hs_tl_cdich_N
one', 'hs_tl_mdich_None', 'h_humidity_preg_None', 'h_pressure_preg_None', 'h_temperature_preg_None', 'hs_hum_mt_hs_h_None', 'hs_tm_mt_hs_h_None', 'hs_uvdvf_mt_hs_h_None', 'hs_hum_dy_hs_h_None', 'hs_hum_wk_hs_h_None', 'hs_tm_dy_hs_h_None', 'hs_tm_
wk_hs_h_None', 'hs_uvdvf_dy_hs_h_None', 'hs_uvdvf_wk_hs_h_None', 'hs_blueyn300_s_None', 'h_blueyn300_preg_None', 'h_greenyn300_preg_None', 'h_ndvi100_preg_None', 'hs_greenyn300_s_None', 'hs_blueyn300_h_None', 'hs_greenyn300_h_None', 'hs_ndvi100_h
_None', 'hs_ndvi100_s_None', 'h_lden_cat_preg_None', 'hs_ln_cat_h_None', 'hs_lden_cat_s_None', 'hs_dde_cadj_Log2', 'hs_dde_madj_Log2', 'hs_ddt_cadj_Log2', 'hs_ddt_madj_Log2', 'hs_hcb_cadj_Log2', 'hs_hcb_madj_Log2', 'hs_pcb118_cadj_Log2', 'hs_pcb1
18_madj_Log2', 'hs_pcb138_cadj_Log2', 'hs_pcb138_madj_Log2', 'hs_pcb153_cadj_Log2', 'hs_pcb153_madj_Log2', 'hs_pcb170_cadj_Log2', 'hs_pcb170_madj_Log2', 'hs_pcb180_cadj_Log2', 'hs_pcb180_madj_Log2', 'hs_sumPCBs5_cadj_Log2', 'hs_sumPCBs5_madj_Log
2', 'hs_dep_cadj_Log2', 'hs_dep_madj_Log2', 'hs_detp_cadj_Log2', 'hs_detp_madj_Log2', 'hs_dmdtp_cdich_None', 'hs_dmp_cadj_Log2', 'hs_dmp_madj_Log2', 'hs_dmtp_cadj_Log2', 'hs_dmtp_madj_Log2', 'hs_pbde153_cadj_Log2', 'hs_pbde153_madj_Log2', 'hs_pbd
e47_cadj_Log2', 'hs_pbde47_madj_Log2', 'hs_pfhxs_c_Log2', 'hs_pfhxs_m_Log2', 'hs_pfna_c_Log2', 'hs_pfna_m_Log2', 'hs_pfoa_c_Log2', 'hs_pfoa_m_Log2', 'hs_pfos_c_Log2', 'hs_pfos_m_Log2', 'hs_pfunda_c_Log2', 'hs_pfunda_m_Log2', 'hs_bpa_cadj_Log2',
'hs_bpa_madj_Log2', 'hs_bupa_cadj_Log2', 'hs_bupa_madj_Log2', 'hs_etpa_cadj_Log2', 'hs_etpa_madj_Log2', 'hs_mepa_cadj_Log2', 'hs_mepa_madj_Log2', 'hs_oxbe_cadj_Log2', 'hs_oxbe_madj_Log2', 'hs_prpa_cadj_Log2', 'hs_prpa_madj_Log2', 'hs_trcs_cadj_Lo
g2', 'hs_trcs_madj_Log2', 'hs_mbzp_cadj_Log2', 'hs_mbzp_madj_Log2', 'hs_mecpp_cadj_Log2', 'hs_mecpp_madj_Log2', 'hs_mehhp_cadj_Log2', 'hs_mehhp_madj_Log2', 'hs_mehp_cadj_Log2', 'hs_mehp_madj_Log2', 'hs_meohp_cadj_Log2', 'hs_meohp_madj_Log2', 'hs_
mep_cadj_Log2', 'hs_mep_madj_Log2', 'hs_mibp_cadj_Log2', 'hs_mibp_madj_Log2', 'hs_mnbp_cadj_Log2', 'hs_mnbp_madj_Log2', 'hs_ohminp_cadj_Log2', 'hs_ohminp_madj_Log2', 'hs_oxominp_cadj_Log2', 'hs_oxominp_madj_Log2', 'hs_sumDEHP_cadj_Log2', 'hs_sumD
EHP_madj_Log2', 'FAS_cat_None', 'hs_contactfam_3cat_num_None', 'hs_hm_pers_None', 'hs_participation_3cat_None', 'e3_asmokcigd_p_None', 'hs_cotinine_cdich_None', 'hs_cotinine_mcat_None', 'hs_globalexp2_None', 'hs_smk_parents_None', 'h_distinvnear1
_preg_Log', 'h_trafload_preg_pow1over3', 'h_trafnear_preg_pow1over3', 'hs_trafload_h_pow1over3', 'hs_trafnear_h_pow1over3', 'h_bro_preg_Log', 'h_clf_preg_Log', 'h_thm_preg_Log']

🔬 methylome_child Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'SE', 'P.Value', 'adj.P.Val', 'PVAL1', 'PVAL2', 'PVAL3', 'seqnames', 'start', 'end', 'UCSC_RefGene_Name', 'UCSC_RefGene_Group', '2_Effect', '2_P.Value', '2_adj.P.Val', '3_Effec
t', '3_SE', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_SE', '4_P.Value', '4_adj.P.Val']

🔬 methylome_preg Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'SE', 'P.Value', 'adj.P.Val', 'PVAL1', 'PVAL2', 'PVAL3', 'seqnames', 'start', 'end', 'UCSC_RefGene_Name', 'UCSC_RefGene_Group', '2_Effect', '2_SE', '2_P.Value', '2_adj.P.Val',
'3_Effect', '3_SE', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_SE', '4_P.Value', '4_adj.P.Val']

🔬 mirna_child Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'SE', 'P.Value', 'adj.P.Val', 'PVAL1', 'PVAL2', 'PVAL3', '2_Effect', '2_SE', '2_P.Value', '2_adj.P.Val', '3_Effect', '3_SE', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_SE', '4_
P.Value', '4_adj.P.Val', 'MirbaseAccessionNo', 'Gene_Symbol', 'ChromosomeAgilent', 'StartMirbase', 'StopMirbase', 'StrandMirbase', 'CallRate']

🔬 mirna_preg Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'SE', 'P.Value', 'adj.P.Val', 'PVAL1', 'PVAL2', 'PVAL3', '2_Effect', '2_SE', '2_P.Value', '2_adj.P.Val', '3_Effect', '3_SE', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_SE', '4_
P.Value', '4_adj.P.Val', 'MirbaseAccessionNo', 'Gene_Symbol', 'ChromosomeAgilent', 'StartMirbase', 'StopMirbase', 'StrandMirbase', 'CallRate']

🔬 proteins_child Columns:
['index', 'PVAL3', 'P.Value', '3_Effect', '3_P.Value', '2_Effect', '2_P.Value', '2_adj.P.Val', '2_SE', '3_adj.P.Val', 'SE', '4_Effect', 'Effect', '4_P.Value', '4_SE', '4_adj.P.Val', 'PVAL1', 'PVAL2', 'Group', 'Exposure', 'Subgroup', 'Gene_Symbo
l', 'UniProtKB', 'Omics_marker', 'adj.P.Val']

🔬 proteins_preg Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'SE', 'P.Value', 'adj.P.Val', 'UniProtKB', 'Gene_Symbol', 'PVAL1', 'PVAL2', 'PVAL3', '2_SE', '2_P.Value', '2_adj.P.Val', '3_Effect', '3_SE', '3_P.Value', '3_adj.P.Val', '4_Effec
t', '4_SE', '4_P.Value', '4_adj.P.Val']

🔬 metabolites_child Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'P.Value', 'adj.P.Val', 'PVAL1', 'PVAL2', 'PVAL3', '2_Effect', '2_P.Value', '2_adj.P.Val', '3_Effect', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_P.Value', '4_adj.P.Val']

🔬 metabolites_preg Columns:
['index', 'Omics_marker', 'Exposure', 'Period', 'Subgroup', 'Group', 'Effect', 'P.Value', 'adj.P.Val', 'PVAL1', 'PVAL2', 'PVAL3', '2_Effect', '2_P.Value', '2_adj.P.Val', '3_Effect', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_P.Value', '4_adj.P.Va
l']

🔬 transcript_child Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'SE', 'P.Value', 'adj.P.Val', 'seqname', 'strand', 'start', 'stop', 'GeneSymbol_Affy', 'mrna_ID', 'CallRate', 'PVAL1', 'PVAL2', 'PVAL3', '2_Effect', '2_SE', '2_P.Value', '2_adj.
P.Val', '3_Effect', '3_SE', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_SE', '4_P.Value', '4_adj.P.Val']

🔬 transcript_preg Columns:
['index', 'Omics_marker', 'Exposure', 'Subgroup', 'Group', 'Effect', 'SE', 'P.Value', 'adj.P.Val', 'seqname', 'strand', 'start', 'stop', 'GeneSymbol_Affy', 'mrna_ID', 'CallRate', 'PVAL1', 'PVAL2', 'PVAL3', '2_Effect', '2_SE', '2_P.Value', '2_adj.
P.Val', '3_Effect', '3_SE', '3_P.Value', '3_adj.P.Val', '4_Effect', '4_SE', '4_P.Value', '4_adj.P.Val']
```

In [123…

```python
# List of columns that typically contain effect size information
effect_columns_to_check = ['Effect', 'P.Value', 'adj.P.Val']

# For each omics dataset, drop rows where any effect column is NaN or invalid
for name, df in omics_data.items():
    # Keep only the columns that actually exist in this dataframe
    valid_effect_cols = [col for col in effect_columns_to_check if col in df.columns]

    if valid_effect_cols:
        # Drop rows with NaNs in any of those effect columns
        original_len = len(df)
        omics_data[name] = df.dropna(subset=valid_effect_cols)
        new_len = len(omics_data[name])
        print(f"✅ Cleaned '{name}': dropped {original_len - new_len} rows with NaNs in {valid_effect_cols}")
    else:
        print(f"⚠️ No effect columns found in '{name}' - skipping")
```

```
✅ Cleaned 'methylome_child': dropped 100711 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'methylome_preg': dropped 45605 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'mirna_child': dropped 282 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'mirna_preg': dropped 121 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'proteins_child': dropped 0 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'proteins_preg': dropped 0 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'metabolites_child': dropped 0 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'metabolites_preg': dropped 0 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'transcript_child': dropped 15546 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
✅ Cleaned 'transcript_preg': dropped 6776 rows with NaNs in ['Effect', 'P.Value', 'adj.P.Val']
```

In [124…

```python
exposure_mapping = {
    # ✅ Direct Matches
    'h_Absorbance_Log': 'h_Absorbance_Log',
    'h_Benzene_Log': 'h_Benzene_Log',
    'h_NO2_Log': 'h_NO2_Log',
    'h_PM_Log': 'h_PM_Log',
    'h_TEX_Log': 'h_TEX_Log',
    'hs_pfhxs_c_Log2': 'hs_pfhxs_c_Log2',
    'hs_pfhxs_m_Log2': 'hs_pfhxs_m_Log2',
    'hs_pfna_c_Log2': 'hs_pfna_c_Log2',
    'hs_pfna_m_Log2': 'hs_pfna_m_Log2',
    'hs_pfoa_c_Log2': 'hs_pfoa_c_Log2',
    'hs_pfoa_m_Log2': 'hs_pfoa_m_Log2',
    'hs_pfos_c_Log2': 'hs_pfos_c_Log2',
    'hs_pfos_m_Log2': 'hs_pfos_m_Log2',
    'hs_pfunda_c_Log2': 'hs_pfunda_c_Log2',
    'hs_pfunda_m_Log2': 'hs_pfunda_m_Log2',
    'hs_bpa_cadj_Log2': 'hs_bpa_cadj_Log2',
    'hs_bpa_madj_Log2': 'hs_bpa_madj_Log2',
    'hs_bupa_cadj_Log2': 'hs_bupa_cadj_Log2',
    'hs_bupa_madj_Log2': 'hs_bupa_madj_Log2',
    'hs_etpa_cadj_Log2': 'hs_etpa_cadj_Log2',
    'hs_etpa_madj_Log2': 'hs_etpa_madj_Log2',
    'hs_mepa_cadj_Log2': 'hs_mepa_cadj_Log2',
    'hs_mepa_madj_Log2': 'hs_mepa_madj_Log2',
    'hs_oxbe_cadj_Log2': 'hs_oxbe_cadj_Log2',
    'hs_oxbe_madj_Log2': 'hs_oxbe_madj_Log2',
    'hs_prpa_cadj_Log2': 'hs_prpa_cadj_Log2',
    'hs_prpa_madj_Log2': 'hs_prpa_madj_Log2',
    'hs_trcs_cadj_Log2': 'hs_trcs_cadj_Log2',
    'hs_trcs_madj_Log2': 'hs_trcs_madj_Log2',
    'hs_mbzp_cadj_Log2': 'hs_mbzp_cadj_Log2',
    'hs_mbzp_madj_Log2': 'hs_mbzp_madj_Log2',
    'hs_mecpp_cadj_Log2': 'hs_mecpp_cadj_Log2',
    'hs_mecpp_madj_Log2': 'hs_mecpp_madj_Log2',
    'hs_mehhp_cadj_Log2': 'hs_mehhp_cadj_Log2',
    'hs_mehhp_madj_Log2': 'hs_mehhp_madj_Log2',
    'hs_mehp_cadj_Log2': 'hs_mehp_cadj_Log2',
    'hs_mehp_madj_Log2': 'hs_mehp_madj_Log2',
    'hs_meohp_cadj_Log2': 'hs_meohp_cadj_Log2',
    'hs_meohp_madj_Log2': 'hs_meohp_madj_Log2',
    'hs_mep_cadj_Log2': 'hs_mep_cadj_Log2',
    'hs_mep_madj_Log2': 'hs_mep_madj_Log2',
    'hs_mibp_cadj_Log2': 'hs_mibp_cadj_Log2',
    'hs_mibp_madj_Log2': 'hs_mibp_madj_Log2',
    'hs_mnbp_cadj_Log2': 'hs_mnbp_cadj_Log2',
    'hs_mnbp_madj_Log2': 'hs_mnbp_madj_Log2',
    'hs_ohminp_cadj_Log2': 'hs_ohminp_cadj_Log2',
    'hs_ohminp_madj_Log2': 'hs_ohminp_madj_Log2',
    'hs_oxominp_cadj_Log2': 'hs_oxominp_cadj_Log2',
    'hs_oxominp_madj_Log2': 'hs_oxominp_madj_Log2',
    'hs_sumDEHP_cadj_Log2': 'hs_sumDEHP_cadj_Log2',
    'hs_sumDEHP_madj_Log2': 'hs_sumDEHP_madj_Log2',

    # ⚠️ Flagged for Manual Resolution (not found in base file, or ambiguous)
    'h_no2_ratio_preg': 'h_no2_ratio_preg_Log',
    'h_abs_ratio_preg': 'h_abs_ratio_preg_Log',
    'hs_no2_yr_hs_h': 'hs_no2_yr_hs_h_Log',
    'hs_pm25abs_wk_hs_h': 'hs_pm25abs_wk_hs_h_Log',
    'hs_tm_wk_hs_h': 'hs_tm_wk_hs_h_None',
    'hs_pet': 'hs_pet_None',
    'hs_mvpa_prd_alt': 'hs_mvpa_prd_alt_None',
    'hs_as_m': 'hs_as_m_Log2',
    'hs_as_c': 'hs_as_c_Log2',
    'hs_pcb180_cadj': 'hs_pcb180_cadj_Log2',
    'hs_pcb180_madj': 'hs_pcb180_madj_Log2',
}
```

Methylome

In [125…

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# === Step 1: Load Data ===
methylome_child = pd.read_csv("clean_methylome_childhood.csv")
methylome_pregnancy = pd.read_csv("clean_methylome_pregnancy.csv")
base = pd.read_csv("../Helix_extracted.csv")

# === Step 2: Filter methylome data (non-zero, non-null effects + valid p-values) ===
methylome_child = methylome_child[
    (methylome_child['Effect'].notna()) & (methylome_child['Effect'] != 0) &
    (methylome_child['P.Value'].notna()) & (methylome_child['P.Value'] > 0)
]

methylome_pregnancy = methylome_pregnancy[
    (methylome_pregnancy['Effect'].notna()) & (methylome_pregnancy['Effect'] != 0) &
    (methylome_pregnancy['P.Value'].notna()) & (methylome_pregnancy['P.Value'] > 0)
]

# === Step 3: Normalize numeric exposure columns ===
base_z = base.copy()
numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(include=[np.number]).columns
non_numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(exclude=[np.number]).columns

print("✅ Numeric columns to normalize:", len(numeric_cols))
print("🚫 Non-numeric columns skipped:", list(non_numeric_cols))

scaler = StandardScaler()
base_z[numeric_cols] = scaler.fit_transform(base_z[numeric_cols])

# === Step 4: Manual exposure name mapping ===
exposure_map = {
    'Indoor PMabsorbance': 'h_Absorbance_Log',
    'Indoor benzene': 'h_Benzene_Log',
    'Indoor NO2': 'h_NO2_Log',
    'Indoor PM2.5': 'h_PM_Log',
    'Indoor BTEX': 'h_TEX_Log',
    'Accessibility (bus lines 300m)': 'h_accesslines300_preg_dic0',
    'Accessibility (bus stops 300m)': 'h_accesspoints300_preg_Log',
    'Arsenic': 'hs_as_m_Log2',
    'Blue spaces (300 m)': 'h_blueyn300_preg_None',
    'BPA': 'hs_bpa_madj_Log2',
    'Alcohol intake': 'e3_alcpreg_yn_None',
    'Built density (300m)': 'h_builtdens300_preg_Sqrt',
    'Connectivity (300m)': 'h_connind300_preg_Sqrt',
    'Inverse distance to nearest road': 'h_distinvnear1_preg_Log',
    'Water Chloroform': 'h_clf_preg_Log',
    'Water Brominated THMs': 'h_bro_preg_Log'
}

# === Step 5: Risk scoring function ===
def compute_weighted_risk(df, methylome_df, risk_col):
    df[risk_col] = 0.0
    for _, row in methylome_df.iterrows():
        exposure = row["Exposure"]
        effect = row["Effect"]
        pval = row["P.Value"]
        weight = effect * (-np.log10(pval))

        if exposure in exposure_map:
            col = exposure_map[exposure]
            if col in df.columns:
                try:
                    df[risk_col] += df[col].fillna(0).astype(float) * weight
                except Exception as e:
                    print(f"❌ Error with column {col}: {e}")

# === Step 6: Apply to both time points ===
compute_weighted_risk(base_z, methylome_child, "methylome_child_risk")
compute_weighted_risk(base_z, methylome_pregnancy, "methylome_pregnancy_risk")

# === Step 7: Compute total methylome risk ===
base_z["total_methylome_risk"] = base_z["methylome_child_risk"] + base_z["methylome_pregnancy_risk"]

# === Step 8: Save all three risk scores ===
output = base_z[["ID", "methylome_child_risk", "methylome_pregnancy_risk", "total_methylome_risk"]]
output.to_csv("methylome_risk_scores.csv", index=False)
print("✅ Saved: methylome_risk_scores.csv")

# === Step 9: Display top 10 ===
print("\n🔝 Top 10 Children by Total Methylome Risk:")
print(output.sort_values(by="total_methylome_risk", ascending=False).head(10))
```

```
✅ Numeric columns to normalize: 181
🚫 Non-numeric columns skipped: ['h_bfdur_Ter', 'h_cereal_preg_Ter', 'h_dairy_preg_Ter', 'h_fastfood_preg_Ter', 'h_fish_preg_Ter', 'h_fruit_preg_Ter', 'h_legume_preg_Ter', 'h_meat_preg_Ter', 'h_pamod_t3_None', 'h_pavig_t3_None', 'h_veg_preg_Ter',
'hs_bakery_prod_Ter', 'hs_beverages_Ter', 'hs_break_cer_Ter', 'hs_caff_drink_Ter', 'hs_dairy_Ter', 'hs_fastfood_Ter', 'hs_org_food_Ter', 'hs_pet_None', 'hs_proc_meat_Ter', 'hs_readymade_Ter', 'hs_total_bread_Ter', 'hs_total_cereal_Ter', 'hs_total
_fish_Ter', 'hs_total_fruits_Ter', 'hs_total_lipids_Ter', 'hs_total_meat_Ter', 'hs_total_potatoes_Ter', 'hs_total_sweets_Ter', 'hs_total_veg_Ter', 'hs_total_yog_Ter', 'hs_tl_cdich_None', 'hs_tl_mdich_None', 'hs_dmdtp_cdich_None', 'FAS_cat_None',
'hs_contactfam_3cat_num_None', 'hs_participation_3cat_None', 'hs_cotinine_cdich_None', 'hs_cotinine_mcat_None', 'hs_globalexp2_None', 'hs_smk_parents_None']
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2185003731.py:54: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2185003731.py:54: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
✅ Saved: methylome_risk_scores.csv

🔝 Top 10 Children by Total Methylome Risk:
        ID  methylome_child_risk  methylome_pregnancy_risk  \
152    153             18.387822                  7.622379
1076  1077             19.148286                  5.427453
903    904             18.785614                  5.605597
574    575             15.977299                  7.729185
1279  1280             17.801752                  5.653260
1039  1040             16.404161                  7.035990
666    667             17.071854                  5.902272
463    464             15.658210                  7.211443
1135  1136             17.188987                  5.606763
700    701             15.853639                  6.534270

      total_methylome_risk
152              26.010201
1076             24.575739
903              24.391211
574              23.706484
1279             23.455012
1039             23.440152
666              22.974126
463              22.869653
1135             22.795750
700              22.387909
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2185003731.py:74: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  base_z["total_methylome_risk"] = base_z["methylome_child_risk"] + base_z["methylome_pregnancy_risk"]
```

MiRNA

In [126…

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# === Step 1: Load Data ===hood
mirna_child = pd.read_csv("clean_mirna_childhood.csv")
mirna_preg = pd.read_csv("clean_mirna_pregnancy.csv")
base = pd.read_csv("../Helix_extracted.csv")

# === Step 2: Normalize numeric exposure columns ===
base_z = base.copy()
numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(include=[np.number]).columns
non_numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(exclude=[np.number]).columns

print("✅ Numeric columns to normalize:", len(numeric_cols))
print("🚫 Non-numeric columns skipped:", list(non_numeric_cols))

scaler = StandardScaler()
base_z[numeric_cols] = scaler.fit_transform(base_z[numeric_cols])

# === Step 3: Filter miRNA data (non-null effects and p-values + significant FDR) ===
mirna_child = mirna_child[
    (mirna_child['Effect'].notna()) &
    (mirna_child['P.Value'].notna()) &
    (mirna_child['P.Value'] > 0) &
    (mirna_child['adj.P.Val'] < 0.05)
]

mirna_preg = mirna_preg[
    (mirna_preg['Effect'].notna()) &
    (mirna_preg['P.Value'].notna()) &
    (mirna_preg['P.Value'] > 0) &
    (mirna_preg['adj.P.Val'] < 0.05)
]

# === Step 4: Manual exposure name mapping (YOU MUST COMPLETE THIS) ===
mirna_exposure_map = {
    "Accessibility (bus lines 300m)": "h_accesslines300_preg_dic0",
    "Accessibility (bus stops 300m)": "h_accesspoints300_preg_Log",
    "Alcohol intake": "e3_alcpreg_yn_None",
    "Arsenic": "hs_as_m_Log2",
    "BPA": "hs_bpa_madj_Log2",
    "Blue spaces (300 m)": "h_blueyn300_preg_None",
    "Breastfeeding duration (days)": "h_bfdur_Ter",
    "Built density (300m)": "h_builtdens300_preg_Sqrt",
    "Cadmium": "hs_cd_m_Log2",
    "Caffeinated drinks": "hs_caff_drink_Ter",
    "Cat at home": "hs_pet_cat_r2_None",
    "Cereals intake": "h_cereal_preg_Ter",
    "Cesium": "hs_cs_m_Log2",
    "Connectivity (300m)": "h_connind300_preg_Sqrt",
    "Contact with family and friends": "hs_contactfam_3cat_num_None",
    "Copper": "hs_cu_m_Log2",
    "Cotinine": "hs_cotinine_mcat_None",
    "DDE": "hs_dde_madj_Log2",
    "DDT": "hs_ddt_madj_Log2",
    "DEP": "hs_dep_madj_Log2",
    "DETP": "hs_detp_madj_Log2",
    "DMDTP": "hs_dmdtp_cdich_None",
    "DMP": "hs_dmp_madj_Log2",
    "DMTP": "hs_dmtp_madj_Log2",
    "Green spaces (300 m)": "h_greenyn300_preg_None",
    "Humidity (month)": "hs_hum_mt_hs_h_None",
    "Indoor NO2": "h_NO2_Log",
    "Indoor PM2.5": "h_PM_Log",
    "Inverse distance to nearest road": "h_distinvnear1_preg_Log",
    "PCB 118": "hs_pcb118_madj_Log2",
    "PFOA": "hs_pfoa_m_Log2",
    "PFUNDA": "hs_pfunda_m_Log2",
    "TRCS": "hs_trcs_madj_Log2",
    "Temperature (month)": "hs_tm_mt_hs_h_None",
    "UV - Vit.D (month)": "hs_uvdvf_mt_hs_h_None",
    "Water Brominated THMs": "h_bro_preg_Log",
    "Water Chloroform": "h_clf_preg_Log",
    "Water THMs": "h_thm_preg_Log",
    "ETPA": "hs_etpa_madj_Log2"
}


# === Step 5: Risk scoring function ===
def compute_weighted_risk(df, mirna_df, risk_col):
    df[risk_col] = 0.0
    for _, row in mirna_df.iterrows():
        exposure = row["Exposure"]
        effect = row["Effect"]
        pval = row["P.Value"]
        weight = effect * (-np.log10(pval))

        if exposure in mirna_exposure_map:
            col = mirna_exposure_map[exposure]
            if col in df.columns:
                try:
                    df[risk_col] += df[col].fillna(0).astype(float) * weight
                except Exception as e:
                    print(f"❌ Error with column {col}: {e}")
            else:
                print(f"⚠️ Column '{col}' not found in base dataset")
        else:
            print(f"⚠️ Exposure '{exposure}' not mapped")

# === Step 6: Compute child and pregnancy miRNA risk ===
compute_weighted_risk(base_z, mirna_child, "mirna_child_risk")
compute_weighted_risk(base_z, mirna_preg, "mirna_pregnancy_risk")

# === Step 7: Compute total miRNA risk ===
base_z["total_mirna_risk"] = base_z["mirna_child_risk"] + base_z["mirna_pregnancy_risk"]

# === Step 8: Save outputs ===
base_z[["ID", "mirna_child_risk", "mirna_pregnancy_risk", "total_mirna_risk"]].to_csv("mirna_risk_scores.csv", index=False)
print("✅ Saved: mirna_risk_scores.csv")

# === Step 9: Show top 10 by total miRNA risk ===
print("\n🔝 Top 10 Children by Total miRNA Risk:")
print(base_z[["ID", "total_mirna_risk"]].sort_values(by="total_mirna_risk", ascending=False).head(10))
```

```
✅ Numeric columns to normalize: 181
🚫 Non-numeric columns skipped: ['h_bfdur_Ter', 'h_cereal_preg_Ter', 'h_dairy_preg_Ter', 'h_fastfood_preg_Ter', 'h_fish_preg_Ter', 'h_fruit_preg_Ter', 'h_legume_preg_Ter', 'h_meat_preg_Ter', 'h_pamod_t3_None', 'h_pavig_t3_None', 'h_veg_preg_Ter',
'hs_bakery_prod_Ter', 'hs_beverages_Ter', 'hs_break_cer_Ter', 'hs_caff_drink_Ter', 'hs_dairy_Ter', 'hs_fastfood_Ter', 'hs_org_food_Ter', 'hs_pet_None', 'hs_proc_meat_Ter', 'hs_readymade_Ter', 'hs_total_bread_Ter', 'hs_total_cereal_Ter', 'hs_total
_fish_Ter', 'hs_total_fruits_Ter', 'hs_total_lipids_Ter', 'hs_total_meat_Ter', 'hs_total_potatoes_Ter', 'hs_total_sweets_Ter', 'hs_total_veg_Ter', 'hs_total_yog_Ter', 'hs_tl_cdich_None', 'hs_tl_mdich_None', 'hs_dmdtp_cdich_None', 'FAS_cat_None',
'hs_contactfam_3cat_num_None', 'hs_participation_3cat_None', 'hs_cotinine_cdich_None', 'hs_cotinine_mcat_None', 'hs_globalexp2_None', 'hs_smk_parents_None']
⚠️ Exposure 'BUPA' not mapped
⚠️ Exposure 'K' not mapped
⚠️ Exposure 'K' not mapped
⚠️ Exposure 'K' not mapped
⚠️ Exposure 'K' not mapped
⚠️ Exposure 'K' not mapped
⚠️ Exposure 'K' not mapped
⚠️ Exposure 'K' not mapped
⚠️ Exposure 'Na' not mapped
⚠️ Exposure 'Se' not mapped
✅ Saved: mirna_risk_scores.csv

🔝 Top 10 Children by Total miRNA Risk:
        ID  total_mirna_risk
1102  1103         19.488340
418    419         18.162121
809    810         17.239548
325    326         17.148601
1247  1248         16.629678
658    659         14.538093
790    791         14.099200
693    694         13.515083
1012  1013         13.388685
793    794         13.208622
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\3560608163.py:82: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\3560608163.py:82: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\3560608163.py:106: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  base_z["total_mirna_risk"] = base_z["mirna_child_risk"] + base_z["mirna_pregnancy_risk"]
```

Proteins

In [127…

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# === Step 1: Load Data ===
proteins_child = pd.read_csv("clean_proteins_childhood.csv")
proteins_preg = pd.read_csv("clean_proteins_pregnancy.csv")
base = pd.read_csv("../helix_extracted.csv")

# === Step 2: Normalize numeric exposure columns ===
base_z = base.copy()
numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(include=[np.number]).columns
non_numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(exclude=[np.number]).columns

print("✅ Numeric columns to normalize:", len(numeric_cols))
print("🚫 Non-numeric columns skipped:", list(non_numeric_cols))

scaler = StandardScaler()
base_z[numeric_cols] = scaler.fit_transform(base_z[numeric_cols])

# === Step 3: Filter proteins data (non-null effects, valid p-values) ===
proteins_child = proteins_child[
    (proteins_child["Effect"].notna()) &
    (proteins_child["P.Value"].notna()) &
    (proteins_child["P.Value"] > 0)
]

proteins_preg = proteins_preg[
    (proteins_preg["Effect"].notna()) &
    (proteins_preg["P.Value"].notna()) &
    (proteins_preg["P.Value"] > 0)
]

# === Step 4: Define protein exposure-to-column mapping ===
protein_exposure_map = {
    "NO2": "h_NO2_Log",
    "PM2.5": "h_PM_Log",
    "Benzene": "h_Benzene_Log",
    "Absorbance": "h_Absorbance_Log",
    "TEX": "h_TEX_Log",
    "Blue spaces (300 m)": "h_blueyn300_preg_None",
    "Alcohol intake": "e3_alcpreg_yn_None",
    "Cadmium": "hs_cd_m_Log2",
    "Arsenic": "hs_as_m_Log2",
    "Copper": "hs_cu_m_Log2",
    "Cesium": "hs_cs_m_Log2",
    "PCB 118": "hs_pcb118_cadj_Log2",
    "DDE": "hs_dde_cadj_Log2",
    "Humidity (month)": "h_humiditymonth_mean_Log",
    "Temperature (month)": "h_tempmonth_mean_Log",
    "UV - Vit.D (month)": "h_uvvitdmonth_mean_Log",
    "Green spaces (300 m)": "h_greenyn300_preg_None",
    "Water THMs": "h_thms_preg_Log",
    "TRCS": "hs_trcs_cadj_Log2",
    "PFOA": "hs_pfoa_madj_Log2",
    "K": "hs_k_m_Log2",
    "Na": "hs_na_m_Log2",
    "Se": "hs_se_m_Log2",
    "BUPA": "hs_bupa_cadj_Log2"
}

# === Step 5: Scoring Function ===
def compute_weighted_risk(df, protein_df, risk_col):
    df[risk_col] = 0.0
    for _, row in protein_df.iterrows():
        exposure = row["Exposure"]
        effect = row["Effect"]
        pval = row["P.Value"]
        weight = effect * (-np.log10(pval))

        if exposure in protein_exposure_map:
            col = protein_exposure_map[exposure]
            if col in df.columns:
                try:
                    df[risk_col] += df[col].fillna(0).astype(float) * weight
                except Exception as e:
                    print(f"❌ Error with column {col}: {e}")

# === Step 6: Compute child, pregnancy, and total risk scores ===
compute_weighted_risk(base_z, proteins_child, "protein_child_risk")
compute_weighted_risk(base_z, proteins_preg, "protein_pregnancy_risk")
base_z["total_protein_risk"] = base_z["protein_child_risk"] + base_z["protein_pregnancy_risk"]

# === Step 7: Save output ===
base_z[["ID", "protein_child_risk", "protein_pregnancy_risk", "total_protein_risk"]].to_csv("protein_risk_scores.csv", index=False)
print("✅ Protein risk scores saved to protein_risk_scores.csv")

# === Step 8: Display top 10 ===
print("\n🔝 Top 10 Children by Total Protein Risk:")
print(base_z[["ID", "total_protein_risk"]].sort_values(by="total_protein_risk", ascending=False).head(10))
```

```
✅ Numeric columns to normalize: 181
🚫 Non-numeric columns skipped: ['h_bfdur_Ter', 'h_cereal_preg_Ter', 'h_dairy_preg_Ter', 'h_fastfood_preg_Ter', 'h_fish_preg_Ter', 'h_fruit_preg_Ter', 'h_legume_preg_Ter', 'h_meat_preg_Ter', 'h_pamod_t3_None', 'h_pavig_t3_None', 'h_veg_preg_Ter',
'hs_bakery_prod_Ter', 'hs_beverages_Ter', 'hs_break_cer_Ter', 'hs_caff_drink_Ter', 'hs_dairy_Ter', 'hs_fastfood_Ter', 'hs_org_food_Ter', 'hs_pet_None', 'hs_proc_meat_Ter', 'hs_readymade_Ter', 'hs_total_bread_Ter', 'hs_total_cereal_Ter', 'hs_total
_fish_Ter', 'hs_total_fruits_Ter', 'hs_total_lipids_Ter', 'hs_total_meat_Ter', 'hs_total_potatoes_Ter', 'hs_total_sweets_Ter', 'hs_total_veg_Ter', 'hs_total_yog_Ter', 'hs_tl_cdich_None', 'hs_tl_mdich_None', 'hs_dmdtp_cdich_None', 'FAS_cat_None',
'hs_contactfam_3cat_num_None', 'hs_participation_3cat_None', 'hs_cotinine_cdich_None', 'hs_cotinine_mcat_None', 'hs_globalexp2_None', 'hs_smk_parents_None']
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2455461258.py:64: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2455461258.py:64: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
✅ Protein risk scores saved to protein_risk_scores.csv

🔝 Top 10 Children by Total Protein Risk:
        ID  total_protein_risk
405    406           54.113356
621    622           53.133823
1083  1084           52.103328
546    547           51.649371
1001  1002           51.332019
574    575           49.730744
1224  1225           48.090605
1175  1176           46.982884
362    363           46.636949
86      87           46.096314
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2455461258.py:82: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  base_z["total_protein_risk"] = base_z["protein_child_risk"] + base_z["protein_pregnancy_risk"]
```

Transcriptome

In [128…

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# === Step 1: Load Data ===
transcript_child = pd.read_csv("clean_transcriptome_childhood.csv")
transcript_preg = pd.read_csv("clean_transcriptome_pregnancy.csv")
base = pd.read_csv("../helix_extracted.csv")

# === Step 2: Normalize numeric exposure columns ===
base_z = base.copy()
numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(include=[np.number]).columns
scaler = StandardScaler()
base_z[numeric_cols] = scaler.fit_transform(base_z[numeric_cols])

# === Step 3: Filter transcriptome data (non-null effects, valid p-values) ===
transcript_child = transcript_child[
    (transcript_child["Effect"].notna()) &
    (transcript_child["P.Value"].notna()) &
    (transcript_child["P.Value"] > 0)
]

transcript_preg = transcript_preg[
    (transcript_preg["Effect"].notna()) &
    (transcript_preg["P.Value"].notna()) &
    (transcript_preg["P.Value"] > 0)
]

# === Step 4: Exposure mapping (auto-mapped only) ===
transcript_exposure_map = {
    'Accessibility (bus lines 300m)': 'h_accesslines300_preg_dic0',
    'Accessibility (bus stops 300m)': 'h_accesspoints300_preg_Log',
    'Alcohol intake': 'e3_alcpreg_yn_None',
    'Arsenic': 'hs_as_m_Log2',
    'Blue spaces (300 m)': 'h_blueyn300_preg_None',
    'Built density (300m)': 'h_builtdens300_preg_Sqrt',
    'Connectivity (300m)': 'h_connind300_preg_Sqrt',
    'Green spaces (300 m)': 'h_greenyn300_preg_None',
    'Humidity (month)': 'h_humidity_preg_None',
    'Indoor BTEX': 'h_TEX_Log',
    'Indoor NO2': 'h_NO2_Log',
    'Indoor PM2.5': 'h_PM_Log',
    'Indoor PMabsorbance': 'h_Absorbance_Log',
    'Indoor benzene': 'h_Benzene_Log',
    'Inverse distance to nearest road': 'h_distinvnear1_preg_Log',
    'Moderate and vigorous PA': 'hs_mvpa_prd_alt_None',
    'NDVI (100 m)': 'h_ndvi100_preg_None',
    'OH-MiNP': 'hs_ohminp_madj_Log2',
    'PM10 (year)': 'hs_pm10_yr_hs_h_None',
    'PM2.5 (year)': 'hs_pm25_yr_hs_h_None',
    'PMabsorbance (year)': 'hs_pm25abs_yr_hs_h_Log',
    'Parental smoking': 'hs_smk_parents_None',
    'Processed meat intake': 'hs_proc_meat_Ter',
    'Sedentary behaviour': 'hs_sd_wk_None',
    'Social participation': 'hs_participation_3cat_None',
    'Temperature (month)': 'h_temperature_preg_None',
    'Traffic noise (night)': 'hs_ln_cat_h_None',
    'Traffic noise (24h)': 'h_lden_cat_preg_None',
    'UV - Vit.D (month)': 'hs_uvdvf_mt_hs_h_None',
}

# === Step 5: Scoring function ===
def compute_transcript_risk(df, omics_df, exposure_map, risk_col):
    df[risk_col] = 0.0
    for _, row in omics_df.iterrows():
        exposure = row["Exposure"]
        effect = row["Effect"]
        pval = row["P.Value"]
        weight = effect * (-np.log10(pval))

        if exposure in exposure_map:
            col = exposure_map[exposure]
            if col in df.columns and pd.api.types.is_numeric_dtype(df[col]):
                try:
                    df[risk_col] += df[col].fillna(0).astype(float) * weight
                except Exception as e:
                    print(f"❌ Error with {col}: {e}")


# === Step 6: Compute risk scores ===
compute_transcript_risk(base_z, transcript_child, transcript_exposure_map, "transcript_child_risk")
compute_transcript_risk(base_z, transcript_preg, transcript_exposure_map, "transcript_pregnancy_risk")
base_z["total_transcript_risk"] = base_z["transcript_child_risk"] + base_z["transcript_pregnancy_risk"]

# === Step 7: Save output ===
base_z[["ID", "transcript_child_risk", "transcript_pregnancy_risk", "total_transcript_risk"]].to_csv("transcript_risk_scores.csv", index=False)
print("✅ Transcriptome risk scores saved to transcript_risk_scores.csv")

# === Step 8: Display top 10 ===
print("\n🔝 Top 10 Children by Total Transcriptome Risk:")
print(base_z[["ID", "total_transcript_risk"]].sort_values(by="total_transcript_risk", ascending=False).head(10))
```

```
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\3416895471.py:64: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\3416895471.py:64: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
✅ Transcriptome risk scores saved to transcript_risk_scores.csv

🔝 Top 10 Children by Total Transcriptome Risk:
        ID  total_transcript_risk
1130  1131              97.390014
1134  1135              92.841944
1093  1094              92.713022
362    363              91.012738
390    391              90.294902
711    712              89.484796
22      23              86.155272
611    612              85.794268
669    670              80.864301
819    820              78.194139
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\3416895471.py:83: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  base_z["total_transcript_risk"] = base_z["transcript_child_risk"] + base_z["transcript_pregnancy_risk"]
```

Serum Metabolites

In [129…

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# === Step 1: Load Data ===
metabo_child = pd.read_csv("clean_serum_metabolites_childhood.csv")
metabo_preg = pd.read_csv("clean_serum_metabolites_pregnancy.csv")
base = pd.read_csv("../helix_extracted.csv")

# === Step 2: Normalize numeric exposure columns ===
base_z = base.copy()
numeric_cols = base_z.drop(columns=["ID"]).select_dtypes(include=[np.number]).columns
scaler = StandardScaler()
base_z[numeric_cols] = scaler.fit_transform(base_z[numeric_cols])

# === Step 3: Filter metabolite data (non-null effects, valid p-values) ===
metabo_child = metabo_child[
    (metabo_child["Effect"].notna()) &
    (metabo_child["P.Value"].notna()) &
    (metabo_child["P.Value"] > 0)
]

metabo_preg = metabo_preg[
    (metabo_preg["Effect"].notna()) &
    (metabo_preg["P.Value"].notna()) &
    (metabo_preg["P.Value"] > 0)
]

# === Step 4: Exposure mapping (auto-mapped only) ===
metabo_exposure_map = {
    'Accessibility (bus lines 300m)': 'h_accesslines300_preg_dic0',
    'Accessibility (bus stops 300m)': 'h_accesspoints300_preg_Log',
    'Alcohol intake': 'e3_alcpreg_yn_None',
    'Arsenic': 'hs_as_m_Log2',
    'BPA': 'hs_bpa_cadj_Log2',
    'BUPA': 'hs_bupa_cadj_Log2',
    'Blue spaces (300 m)': 'h_blueyn300_preg_None',
    'Built density (300m)': 'h_builtdens300_preg_Sqrt',
    'Cadmium': 'hs_cd_m_Log2',
    'Cesium': 'hs_cs_m_Log2',
    'Cobalt': 'hs_co_m_Log2',
    'Connectivity (300m)': 'h_connind300_preg_Sqrt',
    'Copper': 'hs_cu_m_Log2',
    'Cotinine': 'hs_cotinine_cdich_None',
    'DDE': 'hs_dde_cadj_Log2',
    'DDT': 'hs_ddt_madj_Log2',
    'DEP': 'hs_dep_madj_Log2',
    'DETP': 'hs_detp_cadj_Log2',
    'DMDTP': 'hs_dmdtp_cdich_None',
    'DMP': 'hs_dmp_madj_Log2',
    'DMTP': 'hs_dmtp_cadj_Log2',
    'ETPA': 'hs_etpa_madj_Log2',
    'Green spaces (300 m)': 'h_greenyn300_preg_None',
    'HCB': 'hs_hcb_cadj_Log2',
    'Humidity (month)': 'h_humidity_preg_None',
    'Indoor BTEX': 'h_TEX_Log',
    'Indoor NO2': 'h_NO2_Log',
    'Indoor PM2.5': 'h_PM_Log',
    'Indoor PMabsorbance': 'h_Absorbance_Log',
    'Indoor benzene': 'h_Benzene_Log',
    'Inverse distance to nearest road': 'h_distinvnear1_preg_Log',
    'K': 'hs_k_m_Log2',
    'MECPP': 'hs_mecpp_cadj_Log2',
    'MEHHP': 'hs_mehhp_cadj_Log2',
    'MEHP': 'hs_mehp_cadj_Log2',
    'MEOHP': 'hs_meohp_madj_Log2',
    'MEP': 'hs_mep_madj_Log2',
    'MEPA': 'hs_mepa_cadj_Log2',
    'MBzP': 'hs_mbzp_madj_Log2',
    'MiBP': 'hs_mibp_madj_Log2',
    'MnBP': 'hs_mnbp_madj_Log2',
    'Na': 'hs_na_m_Log2',
    'NDVI (100 m)': 'h_ndvi100_preg_None',
    'OH-MiNP': 'hs_ohminp_madj_Log2',
    'OXBE': 'hs_oxbe_cadj_Log2',
    'Parental smoking': 'hs_smk_parents_None',
    'PFOA': 'hs_pfoa_madj_Log2',
    'Se': 'hs_se_m_Log2',
    'Social participation': 'hs_participation_3cat_None',
    'Temperature (month)': 'h_temperature_preg_None',
    'Traffic noise (24h)': 'h_lden_cat_preg_None',
    'Traffic noise (night)': 'hs_ln_cat_h_None',
    'TRCS': 'hs_trcs_cadj_Log2',
    'UV - Vit.D (month)': 'hs_uvdvf_mt_hs_h_None',
    'Walkability index': 'h_walkability_mean_preg_None',
    'Zinc': 'hs_zn_m_Log2'
}

# === Step 5: Risk scoring function ===
def compute_metabo_risk(df, omics_df, exposure_map, risk_col):
    df[risk_col] = 0.0
    for _, row in omics_df.iterrows():
        exposure = row["Exposure"]
        effect = row["Effect"]
        pval = row["P.Value"]
        weight = effect * (-np.log10(pval))

        if exposure in exposure_map:
            col = exposure_map[exposure]
            if col in df.columns and pd.api.types.is_numeric_dtype(df[col]):
                try:
                    df[risk_col] += df[col].fillna(0).astype(float) * weight
                except Exception as e:
                    print(f"❌ Error with {col}: {e}")

# === Step 6: Apply risk scoring ===
compute_metabo_risk(base_z, metabo_child, metabo_exposure_map, "metabo_child_risk")
compute_metabo_risk(base_z, metabo_preg, metabo_exposure_map, "metabo_pregnancy_risk")
base_z["total_metabo_risk"] = base_z["metabo_child_risk"] + base_z["metabo_pregnancy_risk"]

# === Step 7: Save output ===
base_z[["ID", "metabo_child_risk", "metabo_pregnancy_risk", "total_metabo_risk"]].to_csv("metabolite_risk_scores.csv", index=False)
print("✅ Metabolite risk scores saved to metabolite_risk_scores.csv")

# === Step 8: Show top 10 ===
print("\n🔝 Top 10 Children by Total Metabolite Risk:")
print(base_z[["ID", "total_metabo_risk"]].sort_values(by="total_metabo_risk", ascending=False).head(10))
```

```
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2838125905.py:91: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2838125905.py:91: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  df[risk_col] = 0.0
✅ Metabolite risk scores saved to metabolite_risk_scores.csv

🔝 Top 10 Children by Total Metabolite Risk:
        ID  total_metabo_risk
1006  1007         247.849614
50      51         131.899324
967    968         103.569958
52      53          90.502660
215    216          89.589123
403    404          88.536570
782    783          85.404620
1115  1116          83.443096
1215  1216          82.904236
1272  1273          81.955436
C:\Users\vighn\AppData\Local\Temp\ipykernel_7672\2838125905.py:109: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` many times, which has poor performance.  Consider joining all columns at
once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
  base_z["total_metabo_risk"] = base_z["metabo_child_risk"] + base_z["metabo_pregnancy_risk"]
```

Compiling and Feature Attributes

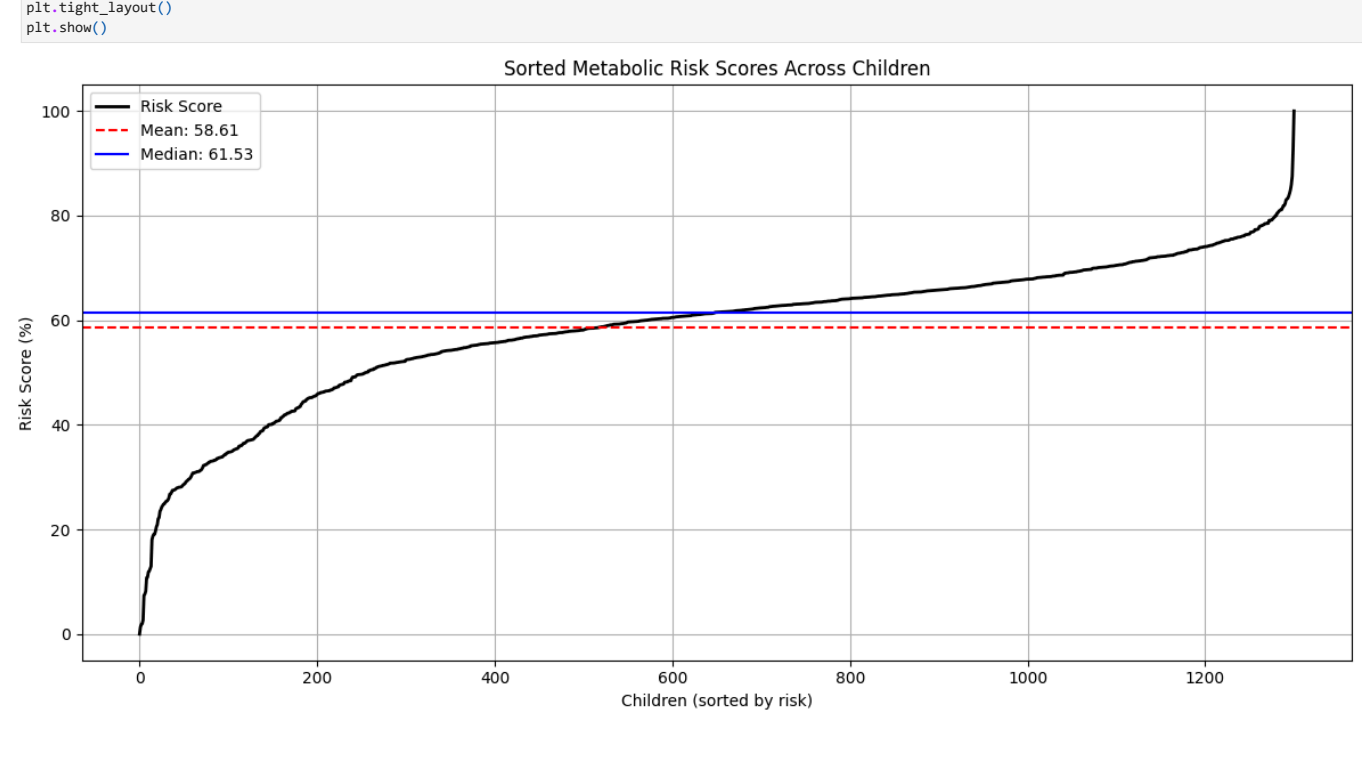
In [130…

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# === Step 1: Load all omics risk score files ===
methylome = pd.read_csv("methylome_risk_scores.csv")
mirna = pd.read_csv("mirna_risk_scores.csv")
proteins = pd.read_csv("protein_risk_scores.csv")
transcript = pd.read_csv("transcript_risk_scores.csv")
metabolites = pd.read_csv("metabolite_risk_scores.csv")

# === Step 2: Merge all datasets on ID ===
df = methylome.merge(mirna, on="ID") \
              .merge(proteins, on="ID") \
              .merge(transcript, on="ID") \
              .merge(metabolites, on="ID")

# === Step 3: Define weights based on literature and regulatory hierarchy ===
weights = {
    "total_methylome_risk": 0.30,
    "total_mirna_risk": 0.15,
    "total_transcript_risk": 0.15,
    "total_protein_risk": 0.20,
    "total_metabo_risk": 0.20
}

# === Step 4: Calculate weighted composite risk ===
df["weighted_composite_risk"] = (
    df["total_methylome_risk"] * weights["total_methylome_risk"] +
    df["total_mirna_risk"] * weights["total_mirna_risk"] +
    df["total_transcript_risk"] * weights["total_transcript_risk"] +
    df["total_protein_risk"] * weights["total_protein_risk"] +
    df["total_metabo_risk"] * weights["total_metabo_risk"]
)

# === Step 5: Normalize composite risk to 0-100% scale ===
scaler = MinMaxScaler()
df["risk_score"] = scaler.fit_transform(df[["weighted_composite_risk"]]) * 100

# === Step 6: Save full output ===
df.to_csv("RISK_weighted.csv", index=False)
print("✅ Final weighted risk scores saved to RISK_weighted.csv")

# === Step 7: View top 10 children by % risk ===
print("\n🔝 Top 10 Children by Weighted Metabolic Risk (%):")
print(df[["ID", "risk_score"]].sort_values(by="risk_score", ascending=False).head(10))
```

```
✅ Final weighted risk scores saved to RISK_weighted.csv

🔝 Top 10 Children by Weighted Metabolic Risk (%):
        ID  risk_score
1006  1007  100.000000
50      51   92.764986
670    671   87.543093
52      53   85.934571
923    924   84.934815
872    873   84.264838
886    887   83.721354
126    127   83.289244
711    712   83.178363
38      39   82.984489
```

In [131…

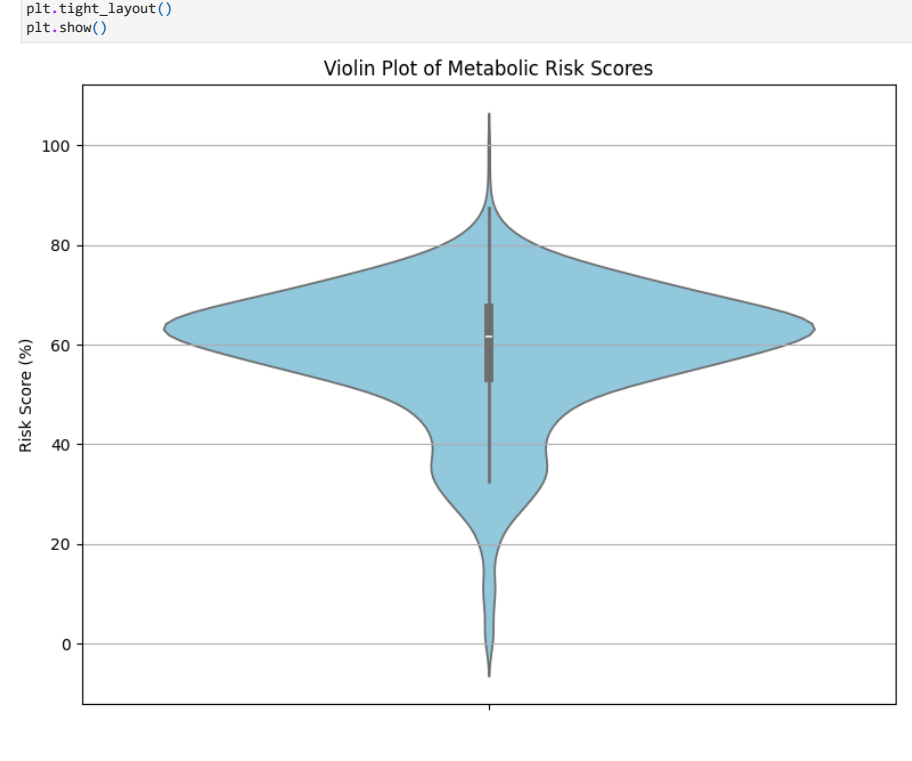
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler


# Load the file
risk = pd.read_csv("RISK_weighted.csv")

# Determine correct risk column
risk_col = "risk_score"  # adjust if needed

# Sort by risk score
risk_sorted = risk.sort_values(by=risk_col).reset_index(drop=True)

# Line plot
plt.figure(figsize=(12, 6))
plt.plot(risk_sorted[risk_col], color='black', linewidth=2, label="Risk Score")

# Mean and median lines
mean_val = risk_sorted[risk_col].mean()
median_val = risk_sorted[risk_col].median()
plt.axhline(mean_val, color='red', linestyle='--', label=f"Mean: {mean_val:.2f}")
plt.axhline(median_val, color='blue', linestyle='-', label=f"Median: {median_val:.2f}")

# Labels and aesthetics
plt.title("Sorted Metabolic Risk Scores Across Children")
plt.xlabel("Children (sorted by risk)")
plt.ylabel("Risk Score (%)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Violin Plot of Risk Score Distribution

In [132…

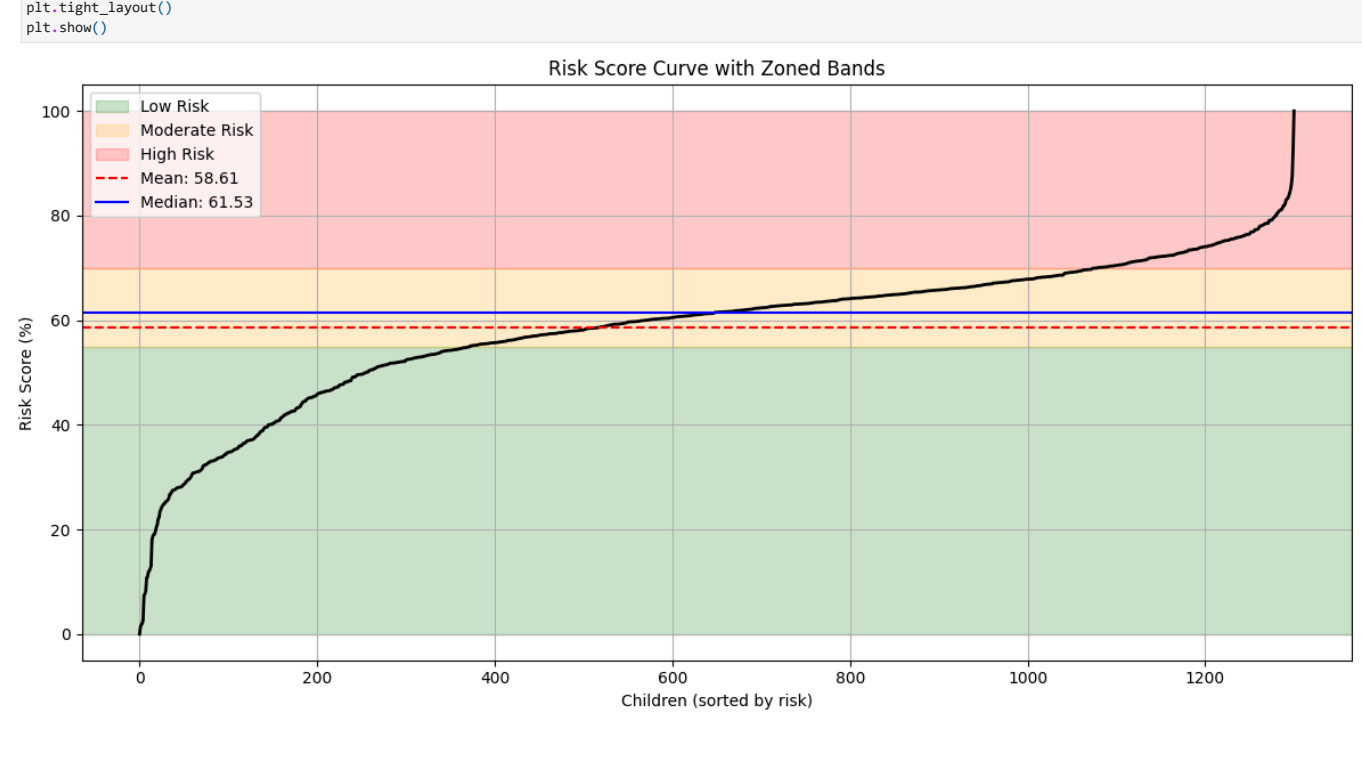
```python
plt.figure(figsize=(8, 6))
sns.violinplot(y=risk["risk_score"], inner="box", color="skyblue")
plt.title("Violin Plot of Metabolic Risk Scores")
plt.ylabel("Risk Score (%)")
plt.grid(True, axis='y')
plt.tight_layout()
plt.show()
```



Zoned Area Plot (Low/Moderate/High Risk Bands)

In [133…

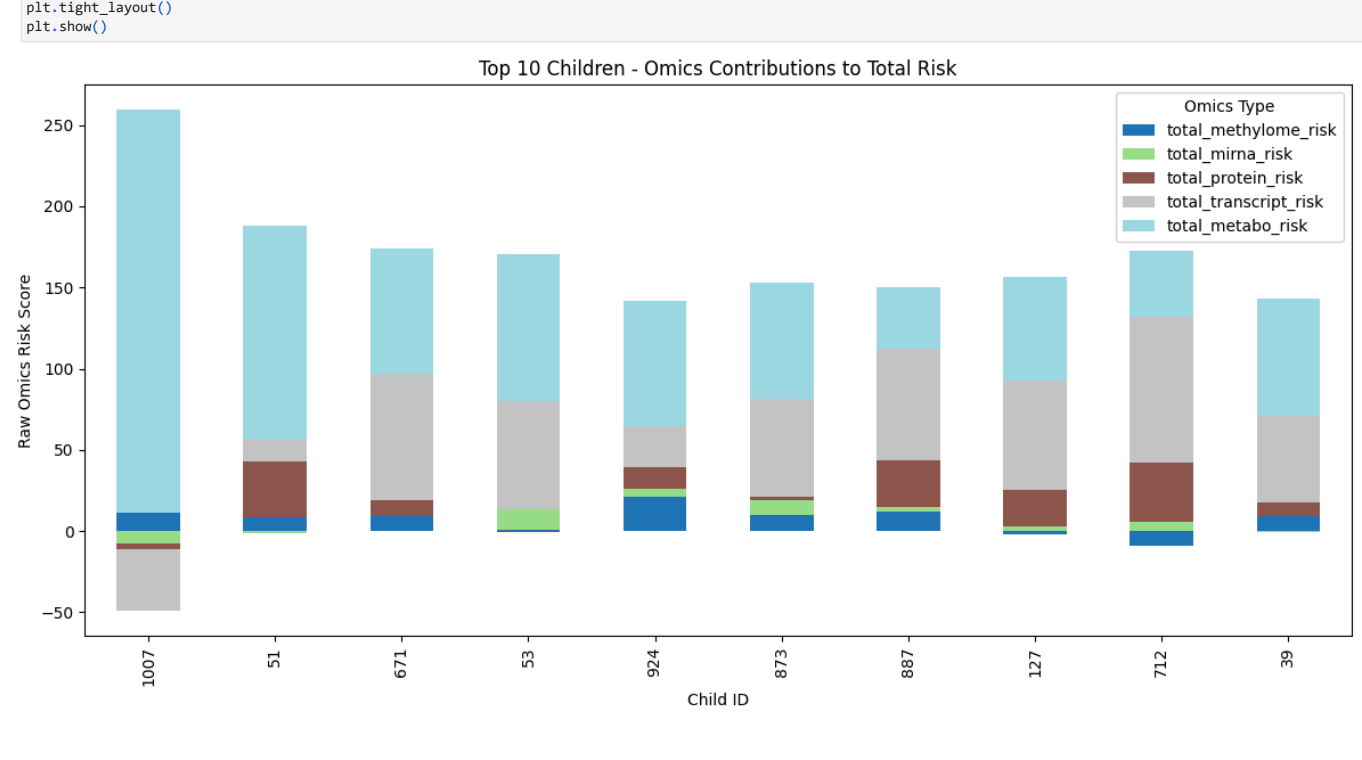
```python
risk_sorted = risk.sort_values(by="risk_score").reset_index(drop=True)

plt.figure(figsize=(12, 6))
plt.plot(risk_sorted["risk_score"], color='black', linewidth=2)

# Background risk zones
plt.axhspan(0, 55, color='green', alpha=0.2, label="Low Risk")
plt.axhspan(55, 70, color='orange', alpha=0.2, label="Moderate Risk")
plt.axhspan(70, 100, color='red', alpha=0.2, label="High Risk")

plt.axhline(risk["risk_score"].mean(), color='red', linestyle='--', label=f"Mean: {risk['risk_score'].mean():.2f}")
plt.axhline(risk["risk_score"].median(), color='blue', linestyle='-', label=f"Median: {risk['risk_score'].median():.2f}")

plt.title("Risk Score Curve with Zoned Bands")
plt.xlabel("Children (sorted by risk)")
plt.ylabel("Risk Score (%)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Stacked Bar Plot: Omics Contributions for Top 10 Children

In [134…

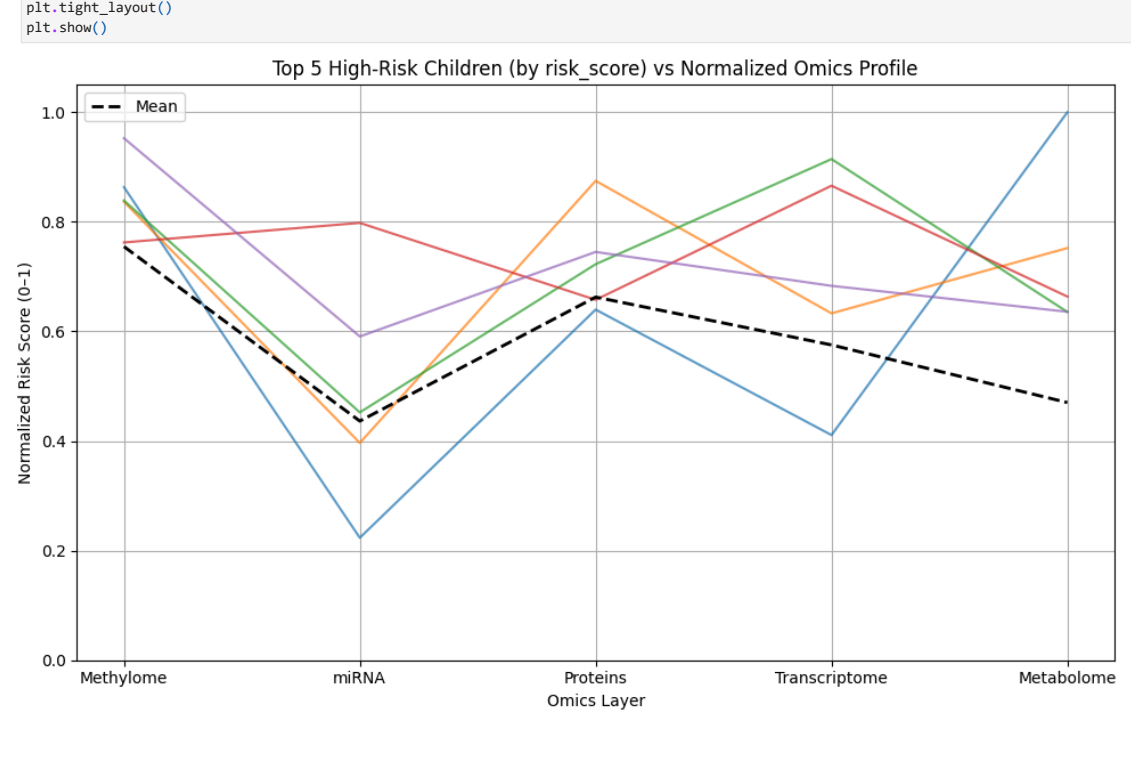
```python
top10 = risk.sort_values(by="risk_score", ascending=False).head(10).copy()
top10.set_index("ID", inplace=True)

omics_cols = [
    "total_methylome_risk", "total_mirna_risk", "total_protein_risk",
    "total_transcript_risk", "total_metabo_risk"
]

top10[omics_cols].plot(kind='bar', stacked=True, figsize=(12, 6), colormap='tab20')
plt.title("Top 10 Children - Omics Contributions to Total Risk")
plt.ylabel("Raw Omics Risk Score")
plt.xlabel("Child ID")
plt.legend(title="Omics Type")
plt.tight_layout()
plt.show()
```



Radar Chart with Scaled Values (0–1)

In [135…

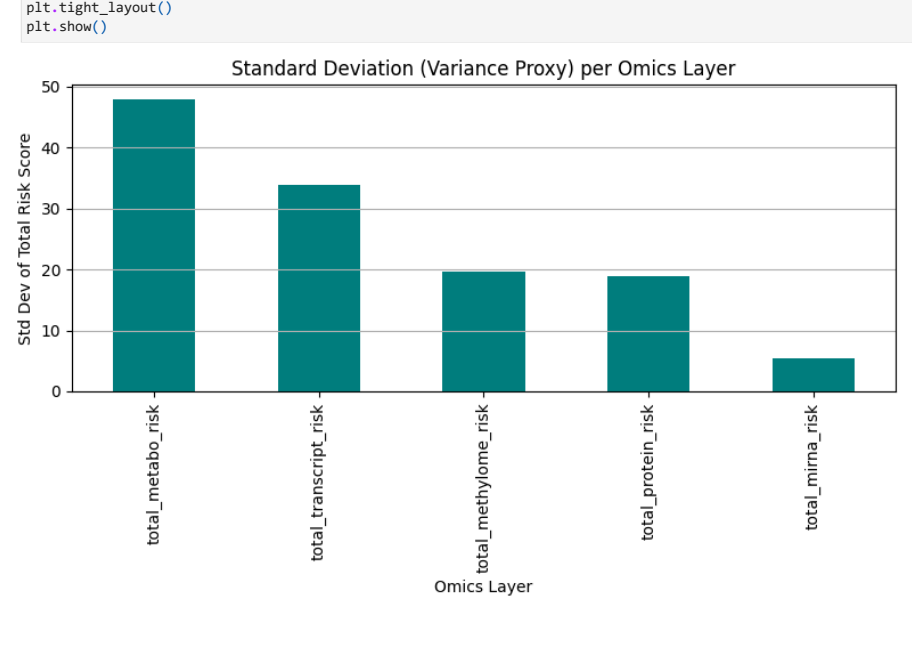
```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

# Load data
risk = pd.read_csv("RISK_weighted.csv")

# Omics columns and labels
omics_cols = [
    "total_methylome_risk", "total_mirna_risk", "total_protein_risk",
    "total_transcript_risk", "total_metabo_risk"
]
labels = ["Methylome", "miRNA", "Proteins", "Transcriptome", "Metabolome"]

# Normalize omics scores to 0-1
scaler = MinMaxScaler()
scaled = scaler.fit_transform(risk[omics_cols])
risk_scaled = pd.DataFrame(scaled, columns=omics_cols)
risk_scaled["ID"] = risk["ID"]
risk_scaled["risk_score"] = risk["risk_score"]  # <- corrected key

# Select top 5 by true risk_score
top5 = risk_scaled.sort_values(by="risk_score", ascending=False).head(5)

# Mean omics profile across full cohort
mean_profile = risk_scaled[omics_cols].mean()

# Plot
plt.figure(figsize=(10, 6))
for _, row in top5.iterrows():
    plt.plot(labels, [row[col] for col in omics_cols], alpha=0.7)

plt.plot(labels, mean_profile.values, color='black', linestyle='--', linewidth=2, label='Mean')

plt.title("Top 5 High-Risk Children (by risk_score) vs Normalized Omics Profile")
plt.ylabel("Normalized Risk Score (0-1)")
plt.xlabel("Omics Layer")
plt.ylim(0, 1.05)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Bar Chart of Omics Variance Explained

In [136…

```python
stds = risk[[
    "total_methylome_risk", "total_mirna_risk", "total_protein_risk",
    "total_transcript_risk", "total_metabo_risk"
]].std()

plt.figure(figsize=(8, 5))
stds.sort_values(ascending=False).plot(kind='bar', color='teal')
plt.title("Standard Deviation (Variance Proxy) per Omics Layer")
plt.ylabel("Std Dev of Total Risk Score")
plt.xlabel("Omics Layer")
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

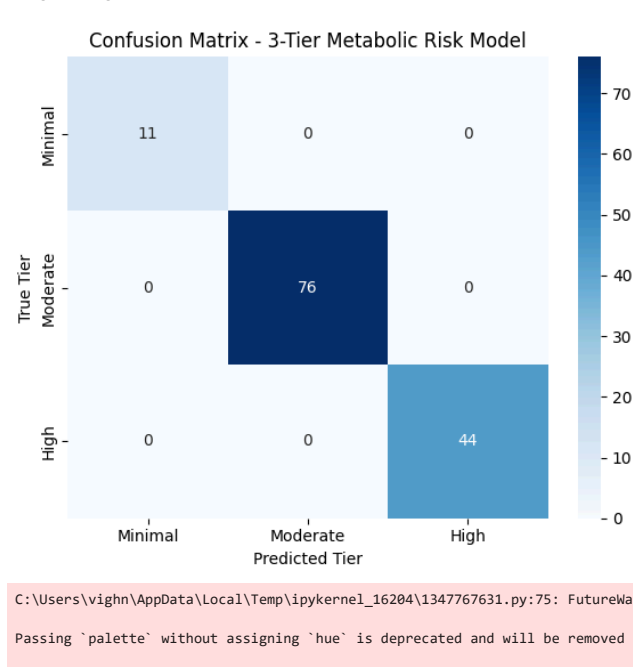


Training and Testing

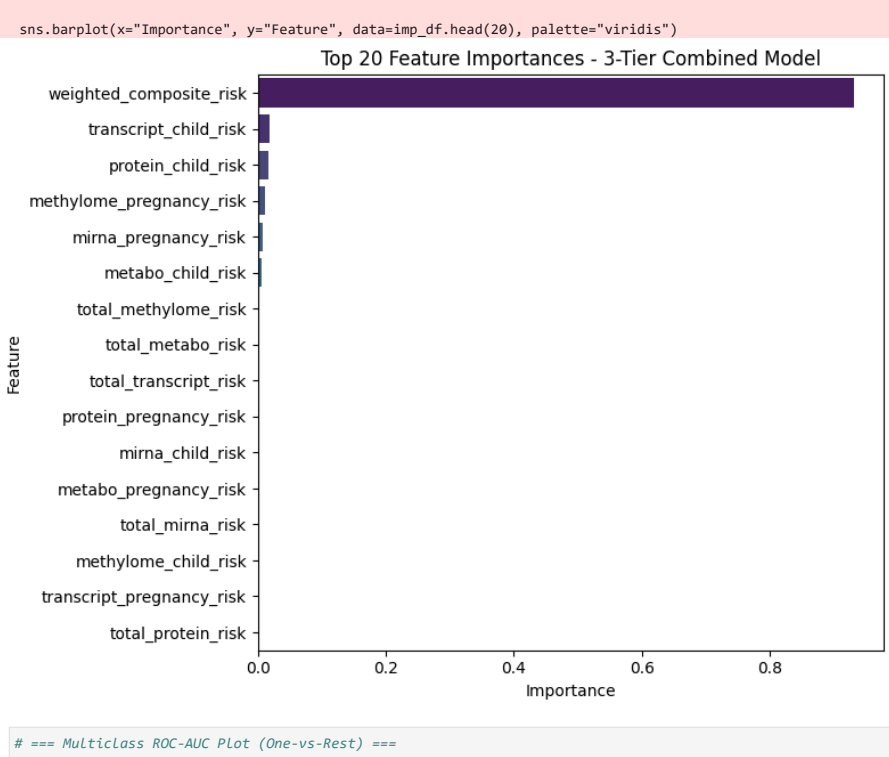
In [4]:

```python
# === 3-Tier Combined Risk Model using XGBoost ===

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# === Step 1: Load risk score data ===
df = pd.read_csv("RISK_weighted.csv")

# === Step 2: Assign categorical risk tiers ===
def assign_risk_tier(score):
    if score < 35:
        return 0  # Minimal Risk
    elif score < 65:
        return 1  # Moderate Risk
    else:
        return 2  # High Risk

df["risk_tier"] = df["risk_score"].apply(assign_risk_tier)

# === Step 3: Define input features and target ===
X = df.drop(columns=["ID", "risk_score", "risk_tier"])
y = df["risk_tier"]

# === Step 4: Stratified 90/10 train/test split ===
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.10, stratify=y, random_state=42
)

# === Step 5: Train XGBoost Classifier for Multiclass ===
model = XGBClassifier(
    objective="multi:softprob",
    num_class=3,
    eval_metric="mlogloss",
    use_label_encoder=False,
    random_state=42
)

model.fit(X_train, y_train)

# === Step 6: Predict and evaluate ===
y_pred = model.predict(X_test)

# Classification report
print("🔍 Classification Report:\n")
print(classification_report(
    y_test, y_pred,
    target_names=["Minimal", "Moderate", "High"]
))

# Confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Minimal", "Moderate", "High"],
            yticklabels=["Minimal", "Moderate", "High"])
plt.title("Confusion Matrix - 3-Tier Metabolic Risk Model")
plt.xlabel("Predicted Tier")
plt.ylabel("True Tier")
plt.tight_layout()
plt.show()

# === Step 7: Feature importance ===
importances = model.feature_importances_
feature_names = X.columns
imp_df = pd.DataFrame({"Feature": feature_names, "Importance": importances})
imp_df = imp_df.sort_values("Importance", ascending=False)

# Top 20 features
plt.figure(figsize=(8, 6))
sns.barplot(x="Importance", y="Feature", data=imp_df.head(20), palette="viridis")
plt.title("Top 20 Feature Importances - 3-Tier Combined Model")
plt.tight_layout()
plt.show()
```

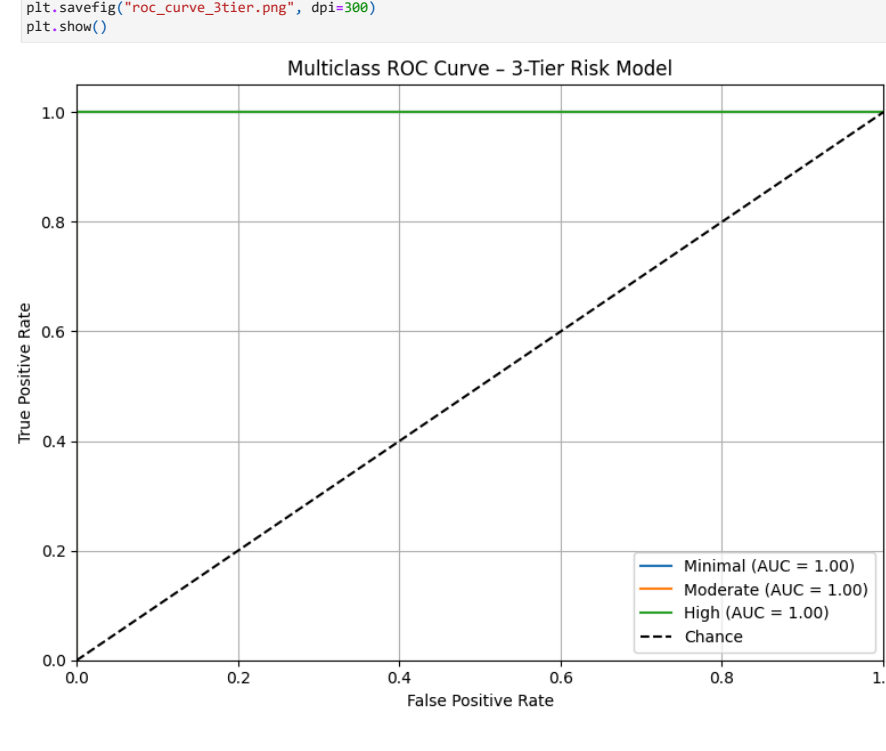
```
c:\Users\vighn\anaconda3\envs\helix_env\lib\site-packages\xgboost\training.py:183: UserWarning: [13:54:52] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
🔍 Classification Report:

              precision    recall  f1-score   support

     Minimal       1.00      1.00      1.00        11
    Moderate       1.00      1.00      1.00        76
        High       1.00      1.00      1.00        44

    accuracy                           1.00       131
   macro avg       1.00      1.00      1.00       131
weighted avg       1.00      1.00      1.00       131
```



```
C:\Users\vighn\AppData\Local\Temp\ipykernel_16204\1347767631.py:75: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x="Importance", y="Feature", data=imp_df.head(20), palette="viridis")
```



In [5]:

```python
# === Multiclass ROC-AUC Plot (One-vs-Rest) ===
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc

# Binarize the output for multiclass ROC
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])
y_score = model.predict_proba(X_test)  # shape = (n_samples, n_classes)

# Compute ROC curve and AUC for each class
fpr = dict()
tpr = dict()
roc_auc = dict()
class_names = ["Minimal", "Moderate", "High"]

for i in range(3):  # for each class
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot all ROC curves
plt.figure(figsize=(8, 6))
for i in range(3):
    plt.plot(fpr[i], tpr[i], label=f"{class_names[i]} (AUC = {roc_auc[i]:.2f})")

plt.plot([0, 1], [0, 1], "k--", label="Chance")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Multiclass ROC Curve - 3-Tier Risk Model")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.savefig("roc_curve_3tier.png", dpi=300)
plt.show()
```